Planning and Reinforcement Learning in AI Systems

(PARLAI)

Tutorial 1 - Decision Making in AI Systems

Guy Azran

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

02360765 Winter 2026-27



AI That Makes Decisions

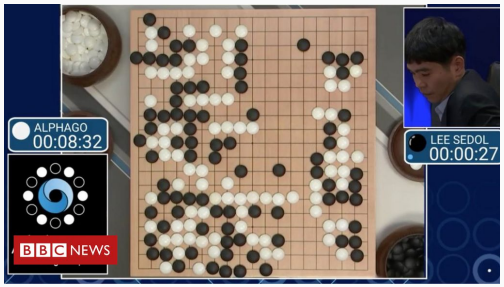
AI Meets the Real World

AI That Makes Decisions

Tutorials in PARLAI

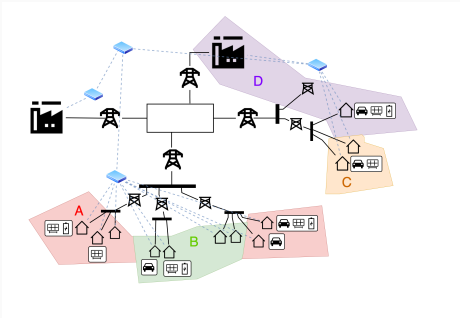
Demo

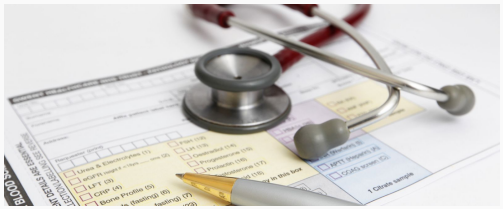
AI systems are no longer just **predicting**. They are **acting**.













Games, driving, logistics, energy, healthcare, manufacturing ...
every one of them is a **sequence of decisions**.

Why Decision Making is Hard (but also exciting!)

- **Decisions have consequences over time.**
  What I do now changes what I can do later, and the reward may only arrive much later.
- **We can't see, know, and predict everything.**
  Observations are partial, actions are noisy, and the world may change on its own.
- **The space of possibilities is huge.**
  Even simple problems have more states than we could ever enumerate.
- **Resources are limited.**
  Time to think, data to learn from, and interactions with the world all cost something.

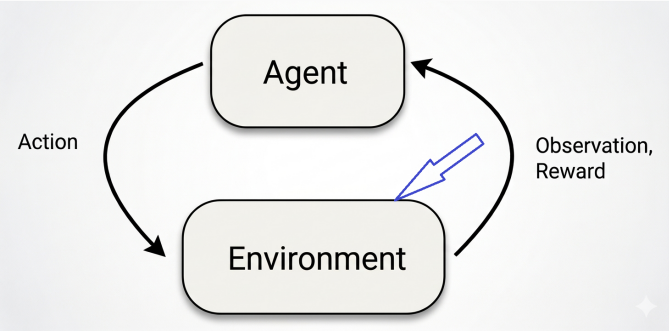
Sequential Decision Making: deciding how to act now to accomplish a long-term objective

AI That Makes Decisions
Tutorials in PARLAI
Demo

Decision Making as an AI Testbed

Decision making is stress-testing intelligence itself. An agent needs to:

- **Observe** an uncertain world.
- **Reason** about goals, uncertainty and the future.
- **Act**, and live with the consequences.
- Over and over again!



Planning

The model of the world is **known**.
Think before you act: search for a good plan.

Reinforcement Learning

The model is **partially known or unknown**.
Learn from experience: act, observe, improve.

AI That Makes Decisions
Tutorials in PARLAI
Demo

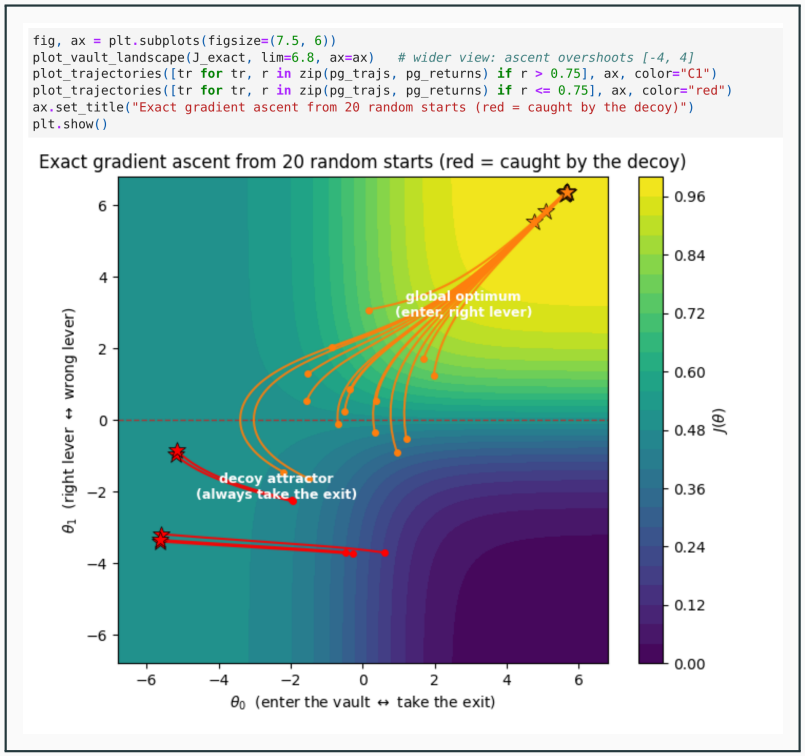
Tutorials in PARLAI

Tutorials in PARLAI

Hands-on Jupyter notebooks that put the lessons into practice:

- Python, JupyterLab & Gymnasium basics
- Imitation learning
- Task planning (PDDL, Fast Downward)
- Monte Carlo Tree Search
- Value-based RL & Deep Q-Networks
- Policy gradient & actor-critic
- Model-based RL

```
fig, ax = plt.subplots(figsize=(7.5, 6))
plot_vault_landscape(J_exact, lim=6.8, ax=ax)   # wider view: ascent overshoots [-4, 4]
plot_trajectories([tr for tr, r in zip(pg_trajs, pg_returns) if r > 0.75], ax, color="C1")
plot_trajectories([tr for tr, r in zip(pg_trajs, pg_returns) if r <= 0.75], ax, color="red")
ax.set_title("Exact gradient ascent from 20 random starts (red = caught by the decoy)")
plt.show()
```



From Grid Worlds to Landing on the Moon

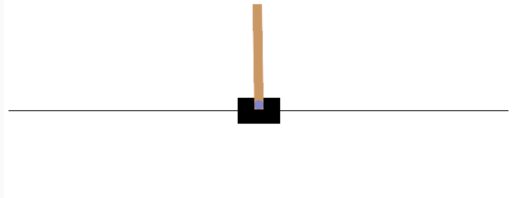
```python
import gymnasium as gym
from PIL import Image

env = gym.make("FrozenLake-v1", is_slippery=False, render_mode="rgb_array")
env.reset()
frame = env.render()
Image.fromarray(frame)
```

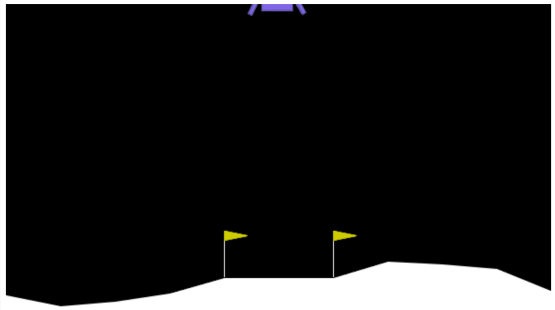


```python
import gymnasium as gym
from PIL import Image

env = gym.make("CartPole-v1", render_mode="rgb_array")
env.reset()
frame = env.render()
Image.fromarray(frame)
```



```python
env = gym.make("LunarLander-v3", render_mode='rgb_array')
env.reset()
frame = env.render()
Image.fromarray(frame)
```



A few lines of code take you from a grid world to a cart-pole to a lunar lander.

AI That Makes Decisions

Tutorials in PARLAI

Demo

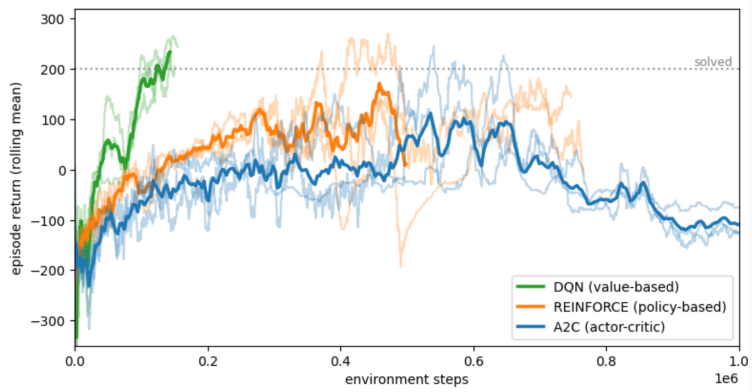
Run It, Watch It, Analyze It

```python
from utils import load_lunar_reference_curves

lunar_ref = load_lunar_reference_curves()

def unified_figure(extra=(), xmax=1_000_000):
    plt.figure(figsize=(9, 4.6))
    plot_seed_curves(lunar_ref["DQN"], color="C2", label="DQN (value-based)")
    plot_seed_curves(lunar_ref["REINFORCE"], color="C1", label="REINFORCE (policy-based)")
    plot_seed_curves([[(s, r) for _, s, r in h] for h in lunar_a2c],
                     color="C0", label="A2C (actor-critic)")
    for runs, color, label in extra:
        plot_seed_curves(runs, color=color, label=label)
    plt.axhline(200, color="gray", linestyle=":", alpha=0.8)
    plt.text(xmax * 0.99, 207, "solved", ha="right", color="gray", fontsize=9)
    plt.xlim(0, xmax)
    plt.ylim(-350, 320)
    plt.xlabel("environment steps")
    plt.ylabel("episode return (rolling mean)")
    plt.legend(loc="lower right")
    plt.show()

unified_figure()
```



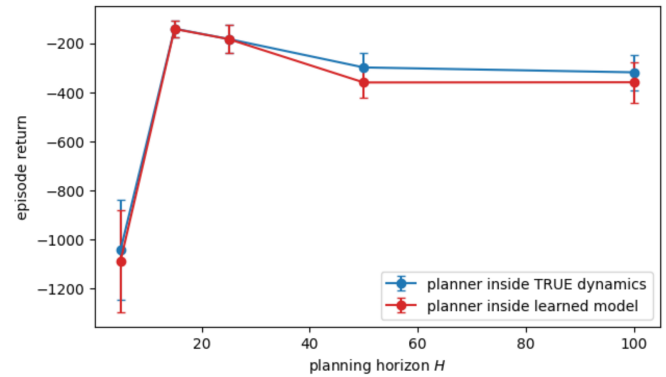

```python
from utils import TrueDyn, TrueRew   # ground-truth Pendulum physics, differentiable

def run_horizon_sweep():
    out = {"learned": {}, "true": {}}
    for H in tqdm([5, 15, 25, 50, 100], desc="horizon"):
        out["learned"][H] = [mpc_episode(pend, dyn, rew, cem_plan, horizon=H, seed=s)
                             for s in range(6)]
        out["true"][H] = [mpc_episode(pend, TrueDyn(), TrueRew(), cem_plan, horizon=H, seed=s)
                          for s in range(6)]
    return out

hor = load_or_run(cache_dir / "mpc_horizon.pkl", run_horizon_sweep, desc="horizon sweep")

plt.figure(figsize=(7, 4))
Hs = sorted(hor["learned"])
for key, color, label in [("true", "C0", "planner inside TRUE dynamics"),
                          ("learned", "C3", "planner inside learned model")]:
    plt.errorbar(Hs, [np.mean(hor[key][H]) for H in Hs],
                 yerr=[np.std(hor[key][H]) / np.sqrt(6) for H in Hs],
                 marker="o", capsize=3, color=color, label=label)
plt.xlabel("planning horizon $H$"); plt.ylabel("episode return"); plt.legend()
plt.show()
```

```
Loaded cached horizon sweep from cache/mpc_horizon.pkl
```



Every tutorial lets you run the algorithms yourself, see what the agents do, and understand why.

Meet Gymnasium

A standard library for building decision-making environments:

- `observation_space`: what the agent can observe
- `action_space`: what the agent can do
- `reset()`: start a new episode
- `step(a)`: act, and get back the next observation and a reward

```python
import gymnasium as gym

env = gym.make("LunarLander-v3")
obs, info = env.reset()

done = False
while not done:
    a = env.action_space.sample()
    obs, r, term, trunc, info = \
        env.step(a)
    done = term or trunc
```

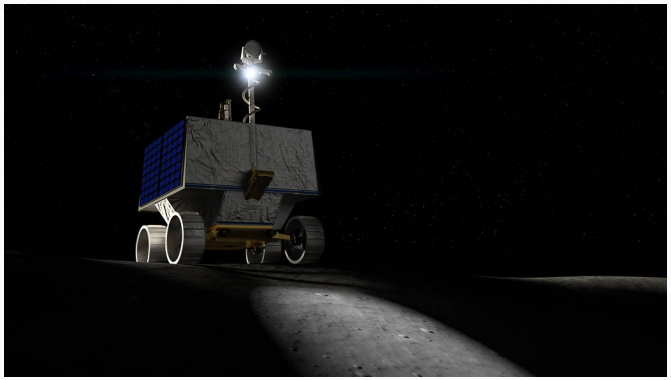
The same interface is used by classic control, Atari, Box2D, MuJoCo robots, and the environments you will build yourselves.

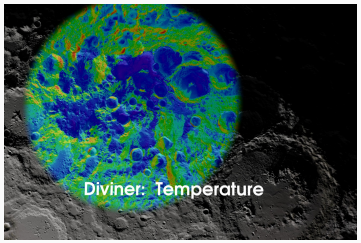
From Primitive Actions to Complex Tasks

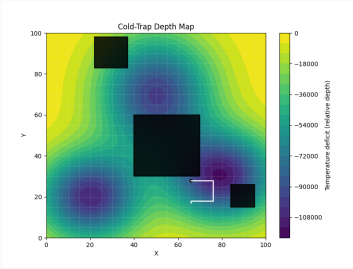
Homework: a lunar cold-trap survey. A rover searches a south-polar crater for the coldest spot, where water ice is most likely to survive. Its battery is limited and its wheels may slip.

Across the assignments you will:

- Model it as a Gymnasium environment
- Solve it with active learning, planning and RL
- Scale up with deep and pre-trained models







Images: NASA

Demo

Demo: Pick-and-Place



Task: clear the table by putting the objects in the basket.

- Which object first?
- How to grasp it?
- What if something is blocking the way?
- What if the grasp fails?

A simple task is a **sequence of decisions**, from high-level planning to low-level control.